

out of this general-purpose server. Depending on the software, the storage solution can exhibit the behaviour of SAN block storage, NAS file storage or even object storage. Ceph, for example, can create all three types of storage out of the same server. In these cases, the disk devices attached to the servers operate as the storage blocks. The disks can be standard direct attached storage (like the one in your laptop) or a number of disks daisy-chained to your server system.

The software defined solutions can be extended and controlled through the software libraries and APIs that they expose. Typically available on a REST API and with UNIX/Linux based operating systems, these are easy to integrate with other orchestration solutions. For example, OpenStack exposes Cinder for block storage, Manila for file storage and Swift for object storage. An application can either run management commands on the natively supported CLI shell or the native/orchestration APIs.

4. The Compute-as-a-Service API

Compute-as-a-Service is the ability to serve the bare metal, the virtual machine or the containers in an on-demand basis over API endpoints or through self-service portals. It is built mostly on top of virtualisation or containerisation platforms. A Compute-as-a-Service model may or may not be a cloud solution. Hypervisors that can be managed through a self-service portal and API endpoint can be considered as Compute-as-a-Service. For example, a VMware vSphere implementation with a self-service portal and API endpoint is such a solution. Similarly, on the containerisation front, the container orchestration tools like Kubernetes are not a cloud solution but a good example of Compute-as-a-Service with an API and self-service GUI. Typical cloud solutions that allow one to provision virtual machines (like AWS EC2), containers (like AWS ECS) and in some cases even physical machines (like Softlayer), are examples of compute power provided as a service.

5. The infrastructure orchestration API

Infrastructure orchestration is the Infrastructure-as-a-Service cloud solution that can offer infrastructure components on demand, as a service, over an API. In case of infrastructure orchestration, it is not only about VM provisioning. It is about orchestrating various infrastructure components in storage, networking and compute, in an optimised manner. This helps provisioning and de-provisioning of components as per the demands of business. The cloud solutions typically offer control over such orchestration through some programming language to configure orchestration logics. For example, AWS provides cloud formation and OpenStack provides the Heat language for this. However, nowadays, in a multi-cloud strategy, new languages have come up for hybrid cloud orchestration. Terraform and Cloudify are two prime examples.

6. Configuration management as code and API

In IT, change and configuration management are the

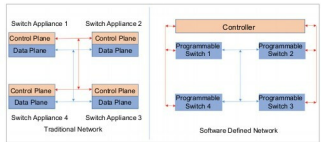


Figure 4: A traditional network vs a software defined network

traditional ITIL processes that track every change in the configuration of systems. Typically, the process is reactive, whereby change is performed on the systems and then recorded in a central configuration management database.

However, currently, changes are first recorded in a database as per the need. Then these changes are applied to systems using automation tools to bring them to the desired state, as recorded in the database. This new-age model is known as the desired state of configuration management. cfEngine, Puppet, Chef, etc, are well known configuration management tools in the market.

These tools configure the target systems as per the desired configuration mentioned in the files. Since this is done by writing text files with a syntax and some logical constructs, these files are known to be infrastructure configuration codes. Using such code to manage infrastructure is known as 'configuration management as code' or 'infrastructure as code'. These tools typically expose an API endpoint to create the desired configuration on target servers.

7. The Platform-as-a-Service API

Platform-as-a-Service (PaaS) solutions provide the platform components such as application, middleware or database, on demand. These solutions hide the complexity of the infrastructure at the backend. At the frontend, they expose a simple GUI or API to provision, de-provision or scale platforms for the application to run.

So instead of saying, "I need a Linux server for installing MySQL," the developer will just have to say, "I need a MySQL instance." In a PaaS solution, deploying a database means it will deploy a new VM, install the required software, open up firewall ports and also provision the other dependencies needed to access the database. It does all of this at the backend, abstracting the complexities from the developers, who only need to ask for the database instance, to get the details. Hence developers can focus on building applications without worrying about the underlying complexities.

The APIs of a PaaS solution can be used by the application to scale itself. Most of the PaaS solutions are based on containers which can run on any VM, be it within the data centre or in the public cloud. So the PaaS solutions can stretch across private and public cloud environments.

Continued on page...40